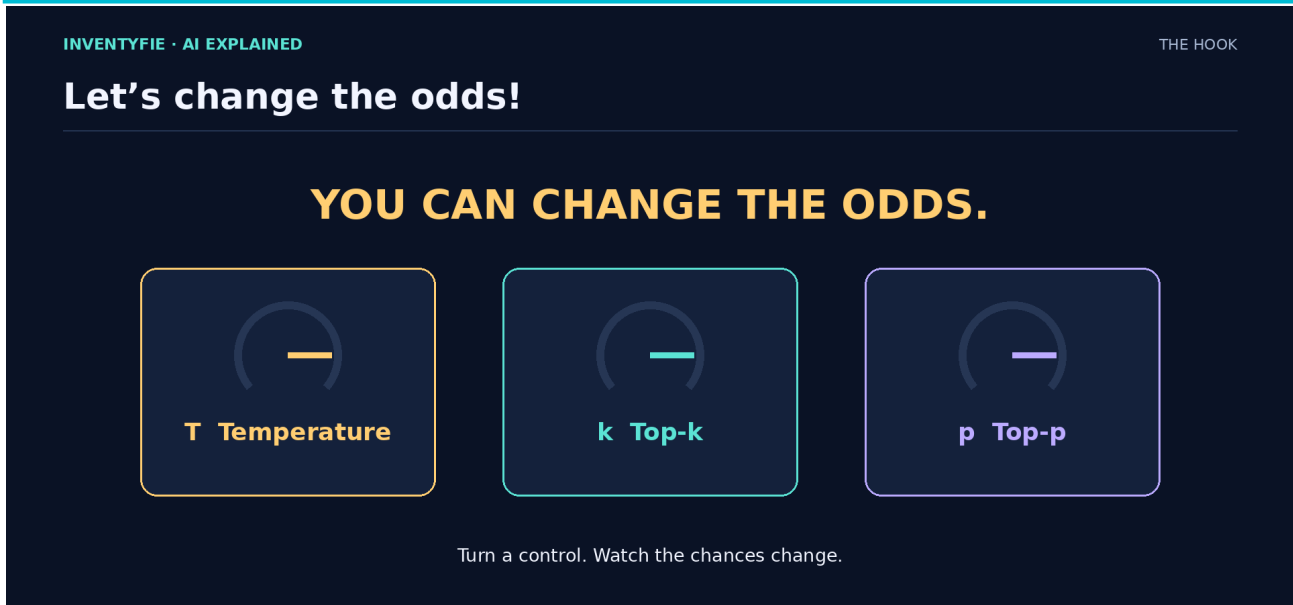


How AI Chooses Its Next Word: Temperature, Top-k and Top-p

Follow an animated experiment to discover how AI turns scores into chances-and how three settings change which token gets picked.



Three controls, three different jobs: reshape the chances, limit the candidate count, and set a probability target.

Watch the accompanying animation, then explore the examples below at your own pace. You only need basic percentages and division to start; the deeper maths is explained step by step.

At a Glance

The question

A dragon opens its mouth. Does it breathe **fire**, **smoke**, or... **bubbles**?

How could an AI model choose different continuations of the same sentence? And what actually changes when you adjust temperature, top-k or top-p?

The simple answer

A large language model gives possible next tokens scores, which are converted into probabilities. **Temperature** changes how concentrated those probabilities are; **top-k** keeps a chosen number of leading candidates; **top-p** keeps enough leading candidates to reach a combined probability target. A random selection then picks one eligible token according to its probability, and the process repeats with the updated text.

Why it matters

- You can understand why an AI sometimes gives familiar wording and sometimes produces a surprising continuation.
- You can distinguish changing a token's chance from removing that token from consideration.

- You can explain why a 90% top-p setting does not mean 90% accuracy-or 90% of the vocabulary.
- You can experiment thoughtfully: change one setting, observe the distribution, and identify what changed.

Think of the candidates as slices on a lottery wheel. A larger slice is easier to land on, but a smaller slice can still win. Temperature changes the slice sizes. Top-k and top-p decide which candidates remain in the lottery. After filtering, the surviving slices are resized to fill one whole wheel again.

Control	Question it answers	Example
Temperature, T	How concentrated should the chances be?	Lower T gives the leading token more probability.
Top-k	How many leading candidates may remain?	k = 3 keeps the top three.
Top-p	How much combined probability should the retained set cover?	p = 0.90 keeps candidates until their total reaches at least 90%.
Sampling	Which eligible token is chosen this time?	Draw one token using the final probabilities.

These controls influence selection, not how much the model knows. A very predictable answer can still be wrong.

About this demonstration: The video uses invented scores and a simplified six-token vocabulary for learning. It does not disclose the private architecture of ChatGPT, Gemini, Claude, or any other product. Available settings and their processing order can differ between implementations.

Deep Dive

1. Start with the core idea

An LLM usually generates text one token at a time. A **token** may be a word, part of a word, punctuation, or another text fragment.

Our example starts with:

The dragon breathed...

We imagine six possible next tokens: fire, smoke, sparks, bubbles, confetti and ice. Treating each as one token keeps the demonstration simple; a real tokenizer may split words differently.

The model does not need to choose the highest-scoring candidate every time. With sampling, several candidates can have a chance. That is why we can hold the model's original scores fixed and still see different selections.

2. Key terms

Term	Meaning in this tutorial
Context	The text available to the model for the current prediction.
Hidden-state vector	A list of numbers representing information the transformer has computed from the context.

Term	Meaning in this tutorial
Weights	Learned numerical parameters used in the model's calculations.
Logit	A raw score for a possible next token, before conversion into probability.
Softmax	A calculation that turns a set of scores into probabilities summing to 100%.
Candidate	A token still eligible to be selected.
Normalization	Rescaling remaining probabilities so they sum to 100%.
Sampling	Randomly selecting a token according to its probability.

Remember: A candidate is one possible choice. Its probability tells you how likely that choice is-not whether the choice is true.

3. How it works

For the combined experiment in this tutorial, follow this sequence:

- Process the context and calculate one logit per candidate token.
- Divide the logits by temperature and convert them into probabilities.
- Keep the top-k candidates and normalize within that remaining pool.
- Apply top-p to that pool and normalize the final retained probabilities.
- Sample one token, append it to the text, and repeat with new context.

This is our teaching sequence. Implementations can combine operations or apply filters differently. First, we will isolate each control so its effect is easy to see.

Where do logits come from?

The transformer computes a hidden-state vector, usually written as **h**. An output projection uses learned weights **W** to convert that vector into token scores:

$$\text{logits} = \mathbf{W} \times \mathbf{h} + \text{optional bias}$$

Each row of **W** combines the numbers in **h** into a score for one vocabulary token. The output weights are learned during training. The hidden state changes with the context, so the resulting logits change too.

Adjusting temperature during generation does not retrain or modify **W**. It scales the resulting scores for selection.

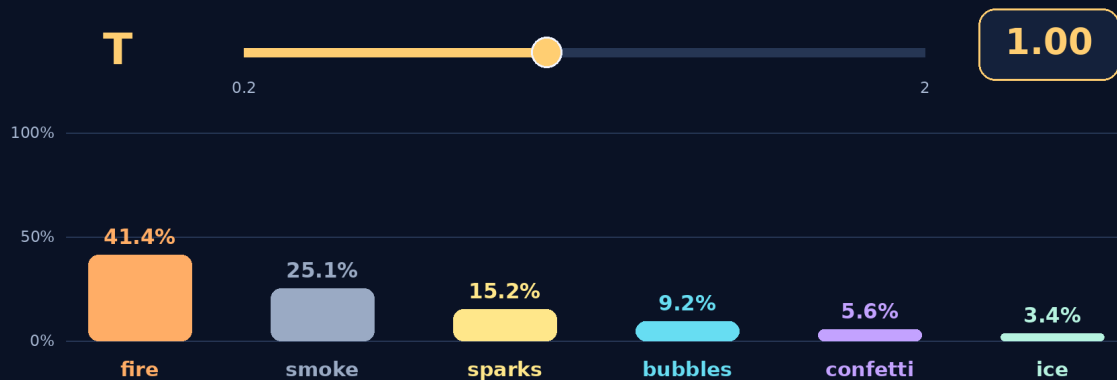
Can logits be negative?

Yes. Logits are scores, not percentages. A logit of -0.5 does not mean a negative probability.

What matters is the gap between scores. The sets $[2, 1, 0]$ and $[-3, -4, -5]$ produce identical softmax probabilities: subtracting five from every score preserves every gap.

4. Try an example: scores become chances

Here's our starting point



Top-k and top-p are disabled. All six tokens remain eligible.

At $T = 1$, fire leads, but every displayed candidate has a chance.

Here are the invented scores used throughout the video:

Token	Original logit	Probability at $T = 1$
fire	2.0	41.41%
smoke	1.5	25.12%
sparks	1.0	15.23%
bubbles	0.5	9.24%
confetti	0.0	5.60%
ice	-0.5	3.40%

Softmax first turns each scaled score into a positive number using the exponential function, $\exp$. It then divides each number by the total:

```
scaled score = logit / T
probability = exp(scaled score) / sum of exp(all scaled scores)
```

For example, $\exp(2)$ is about 7.39, $\exp(0)$ is 1, and $\exp(-0.5)$ is about 0.61. Even a negative score gives a positive exponential. Dividing by the common total creates shares of one whole.

5. Temperature: change the slice sizes

Turn it way down: $T = 0.20$



A bigger slice means a bigger chance of being picked.

At $T = 0.20$, fire occupies about 91.8% of the wheel. A strong preference is still a probability.

Temperature divides the logits **before softmax**. It does not divide the learned weights, and the calculation is $\text{logit} \div \text{temperature}$, not $\text{temperature} \div \text{logit}$.

At $T = 0.5$, fire's score becomes $2 \div 0.5 = 4$, and smoke's becomes $1.5 \div 0.5 = 3$. Their gap doubles from 0.5 to 1. Softmax turns that larger gap into a stronger preference for fire.

Ice's score becomes $-0.5 \div 0.5 = -1$. So "all scores increase" would be incorrect. **The gaps widen.**

Temperature	Fire's probability	Bubbles' probability	What to notice
0.20	91.79%	0.05%	Almost all probability concentrates on fire.
0.50	63.37%	3.15%	Fire becomes more dominant.
1.00	41.41%	9.24%	Our starting distribution.
2.00	28.47%	13.45%	Lower-ranked candidates become more competitive.

For positive T , dividing every score by the same positive number preserves their ranking. Temperature changes the chances, not their order.

If you want the compact mathematical version:

$$P(\text{token } i) = \exp(z_i / T) / \sum_j \exp(z_j / T), \text{ for } T > 0$$

Here z_i is token i 's original logit, and Σ means "add up across candidates." As positive T approaches zero, a unique highest-scoring token approaches 100% probability. $T = 0$ is commonly handled as choosing the highest-scoring token directly, rather than literally dividing by zero.

6. Top-k: choose how many candidates remain

The remaining slices grow



After rescaling, the three surviving chances add up to 100%.

With $k = 3$, fire, smoke and sparks remain. Their probabilities are rescaled to fill the whole wheel.

Reset T to 1. Set k to 3. Keep the three highest-ranked tokens: fire, smoke and sparks. Remove the others from this selection step.

k is a configurable setting where supported. Temperature does not secretly choose it. A service may expose a default, allow you to change it, or not expose it at all.

The retained probabilities originally total about 81.76%. To sample only from those candidates, divide each probability by that retained total:

```
new probability = original probability / total retained probability
```

```
Fire: 0.414085 / 0.817574  $\approx$  0.50648 = 50.65%
```

The new probabilities are approximately 50.65%, 30.72% and 18.63%.

We are rescaling probabilities, not applying softmax to probabilities again. Computing softmax over the retained scaled logits is another way to obtain the same result.

At $k = 1$, only fire remains and its final chance is 100%. Changing temperature cannot bring excluded candidates back. When the context changes at the next prediction step, however, the identities of the top candidates can change.

7. Top-p: reach a combined probability target

90% is a threshold—not 90% of the tokens

$p = 0.90$ · reach 90%



90% target

91.0% covered · 4 tokens · TARGET REACHED!

90% of the probability—not 90% of the vocabulary.

Top-p includes the token that crosses the threshold. A 90% target can therefore retain about 91% before normalization.

Keep T at 1 and disable top-k for this experiment. Set p to 0.90.

Sort candidates from most to least probable, then add their probabilities until the total reaches at least 90%:

Candidates included	Cumulative probability
fire	41.41%
+ smoke	66.52%
+ sparks	81.76%
+ bubbles	91.00% - stop here

Keep these four and normalize their probabilities. Confetti and ice are excluded.

Top-p is also called **nucleus sampling**. It adapts the number of retained candidates to the distribution rather than fixing their count. These definitions of temperature, top-k and top-p are reflected in the Hugging Face generation documentation.

Lower p to 0.65 and two candidates are enough. Raise p to 0.95 and five are needed.

Can the probabilities exceed 100%?

A subset of valid probabilities cannot exceed the full total of 100%. It can exceed the top-p threshold: 91% is above a 90% target, while still below 100%.

Displayed values may have small rounding differences. Keep full precision during calculations and round only for display.

How does temperature affect top-p?

With p fixed at 0.90, our example retains:

- T = 1.00: four candidates.
- T = 0.50: three candidates.

- $T = 0.20$: one candidate, fire.

The p setting stayed the same. Temperature changed the distribution, so a different number of tokens reached the target. This happens automatically from the threshold calculation; nobody changed p .

8. Put T , k and p together

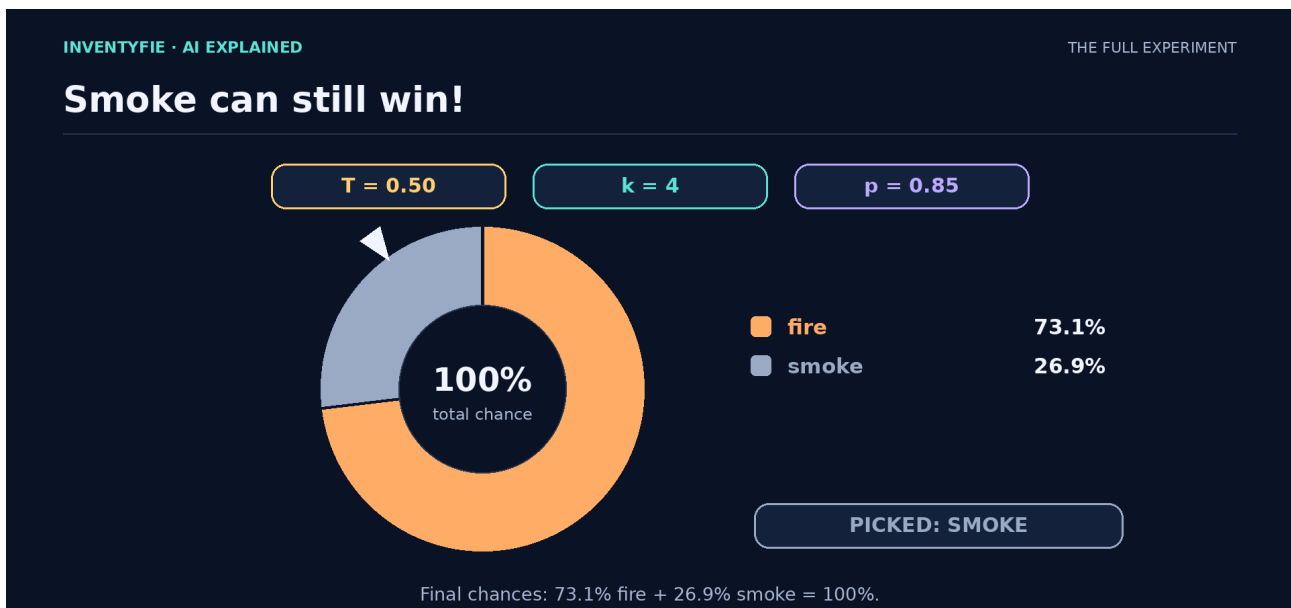
Now try $T = 0.5$, $k = 4$ and $p = 0.85$.

- Temperature gives fire about 63.37% and sharpens the distribution.
- Top- k retains fire, smoke, sparks and bubbles.
- Normalize within those four. The first two now cover about 88.08% of this pool.
- Top- p needs at least 85%, so fire and smoke are enough.
- Normalize again: fire gets about 73.11% and smoke about 26.89%.

If we raise only T to 2, the first three of the top four cover about 83.47% after normalization. That is below 85%, so all four survive top- p this time.

The settings work together, but they have distinct jobs. Our normalization-before-top- p choice matters when interpreting these numbers.

9. The lottery wheel: make the final selection



Fire has about 73% of the final wheel; smoke has about 27%. The smaller slice can still win.

Imagine a pointer that lands uniformly around the wheel. A slice occupying 73% of the wheel has a 73% chance of being selected in one draw—not a promise that it wins.

We can represent that draw with a random number u between 0 and 1:

```
If  $u < 0.7311$ : pick fire.  
Otherwise: pick smoke.
```

```
Illustrative draw:  $u = 0.90$   
Selected token: smoke
```

In the temperature comparison, the video deliberately reuses the same illustrative draw, $u = 0.70$. At $T = 0.20$ it lands on fire; at $T = 2$ it lands on bubbles because the probability boundaries moved. This isolates the effect of changing the distribution.

The model then appends the selected token to the context, calculates new scores and repeats. The wheel represents one selection step, not an entire answer chosen at once.

10. Repetition and hallucination: what these controls cannot promise

Repetition penalties are separate controls that can reduce scores for tokens already used. They encourage alternatives but do not necessarily ban repeated words. Implementations differ, and some repetition is useful: a clear explanation may need to repeat a technical term.

Temperature, top-k and top-p are sampling controls. They do not verify facts or guarantee fewer hallucinations. A low temperature can repeatedly select the same incorrect continuation. Higher temperature can produce unusual wording without adding knowledge.

For factual learning, check important claims against reliable evidence. Keep experimentation with wording separate from checking whether an answer is correct.

11. Check your understanding

If $k = 1$, can increasing T make an excluded token win? No. Only one candidate remains for that step.

Does $p = 0.90$ retain 90% of the vocabulary? No. It retains a leading group covering at least 90% of the relevant probability distribution.

If fire has a 73% chance, must it win the next draw? No. Probability describes a chance, not a promise.

What should you change first when experimenting? One setting at a time. Keep the prompt and other settings fixed, then compare several draws rather than judging from one result.

Takeaway: Temperature reshapes probabilities. Top-k caps the number of candidates. Top-p sets a cumulative probability target. Sampling picks one token-and the process begins again.